



114-2 NCKU Program Design-II HW3

題目說明

我想要設計一個自己的簡易檔案系統 SimpleFS：讓我可以建立資料夾、在裡面放檔案、開啟檔案來讀寫、掛載等，但不需要什麼權限管理功能。

我已經定好了一個程式架構，但是核心程式碼我不會寫，尤其是 `pointer` 的部分，你可以幫助我嗎？

物件說明

檔案系統包含以下物件：

- 節點 (FSNode)
- 檔案系統 (FileSystem)
- 檔案管理器 (FileHandle)

節點 (FSNode)

在 SimpleFS 中，所有的檔案、資料夾都由 FSNode 來記錄，每個 FSNode 代表一個檔案或目錄。每個 FSNode 有一個唯一的 `node_id`，由 FileSystem 在 `allocNode` 時自動指派，不會重複。

使用 `FSNodeType` 決定是檔案或是資料夾：

- `FILE`：代表該 FSNode 為檔案，此處為了簡化，將檔案內容直接紀錄在 `content` 之中，且只能儲存 256 bytes
 - `dentry table` 不應該使用，可以忽略或管理
- `DIRECTORY`：代表該 FSNode 為資料夾，是一種特殊的 FSNode，底下可以存放數個檔案、資料夾，用一張表 (`dentry table`) 記錄。
 - 每一個 `dentry (directory entry)` 紀錄「`name - node id`」的對應關係。當執行 `list()` 列出所有內容時，看到的就是這張表的內容。
 - `content` 不應該使用，可以忽略或管理

每個 `dentry` 的名稱最長為 `MAX_NAME_LEN` (14 個字元)，超過會被截斷。詳細可參見程式模板 `FSNode.h`。

FSNode									
<code>node_id : int</code> <code>type : FSNodeType (FILE DIRECTORY)</code> <code>link_count : int ← # of dentries pointing here</code> <code>open_count : int ← # of FileHandles using this</code>									
<code>if FILE</code> <code>content[256]</code> <code>(file text)</code>	<code>if DIRECTORY</code> <code>dentry_names[16][15]</code> <code>dentry_inums[16]</code> <table><tr><td>0</td><td>"home" 2</td></tr><tr><td>1</td><td>"doc" 5</td></tr><tr><td>2</td><td>(empty)</td></tr><tr><td>..</td><td>...</td></tr></table> <code>mounted_fs → FileSystem*</code>	0	"home" 2	1	"doc" 5	2	(empty)	..	...
0	"home" 2								
1	"doc" 5								
2	(empty)								
..	...								

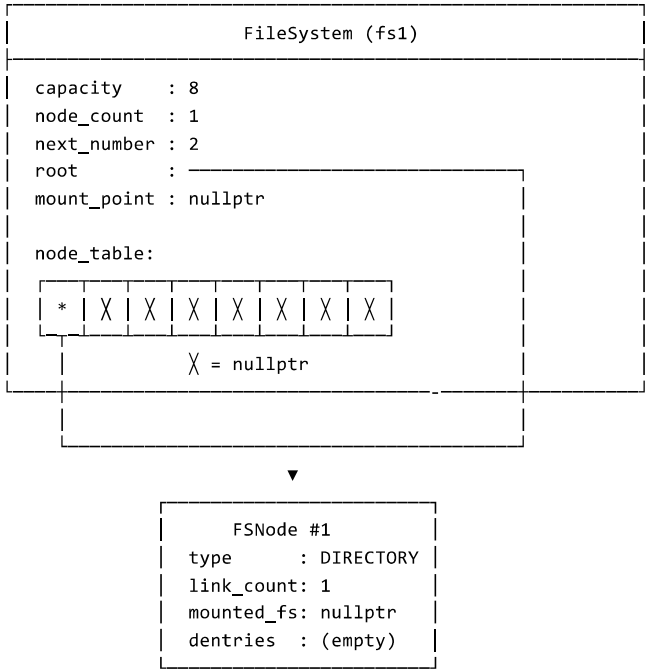
檔案系統 (FileSystem)

檔案系統負責管理所有 FSNode 的生命週期，它決定何時建立 FSNode、何時釋放 FSNode。



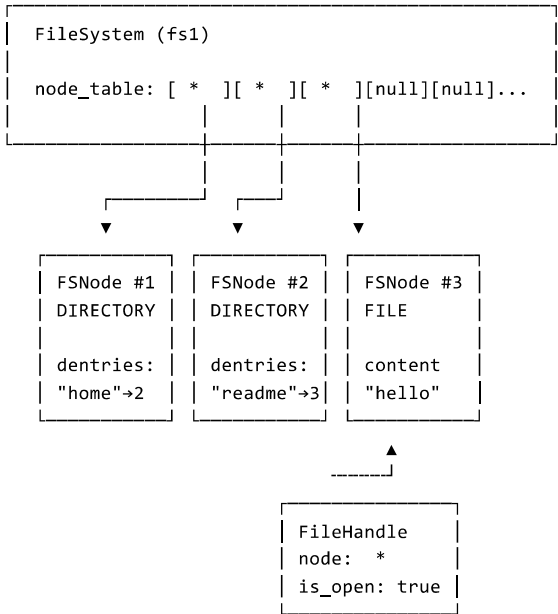
- `node_table` 是固定大小的陣列 (`capacity`)，在建構時決定，模擬真實檔案系統中有限的儲存空間，所以只能存固定數量的 `FSNode`。當節點數量 (`node_count`) 等於 `capacity` 時，代表系統空間已滿，不能再新增檔案或資料夾。
- `next_number` 是下一個要指派的 `node_id`，從 1 開始，每次 `allocNode` 後遞增，確保每個 `FSNode` 的 `id` 唯一。

詳細可參見程式模板 `FileSystem.h`。



檔案管理器 (FileHandle)

`File handle` 負責管理檔案的開啟、讀取、寫入行為。假設此檔案系統的操作都是順序執行，不存在同時寫入的行為，我們也先不考慮多人使用情境。詳細可參見程式模板 `FileHandle.h`。



檔案系統間的掛載 (mount)

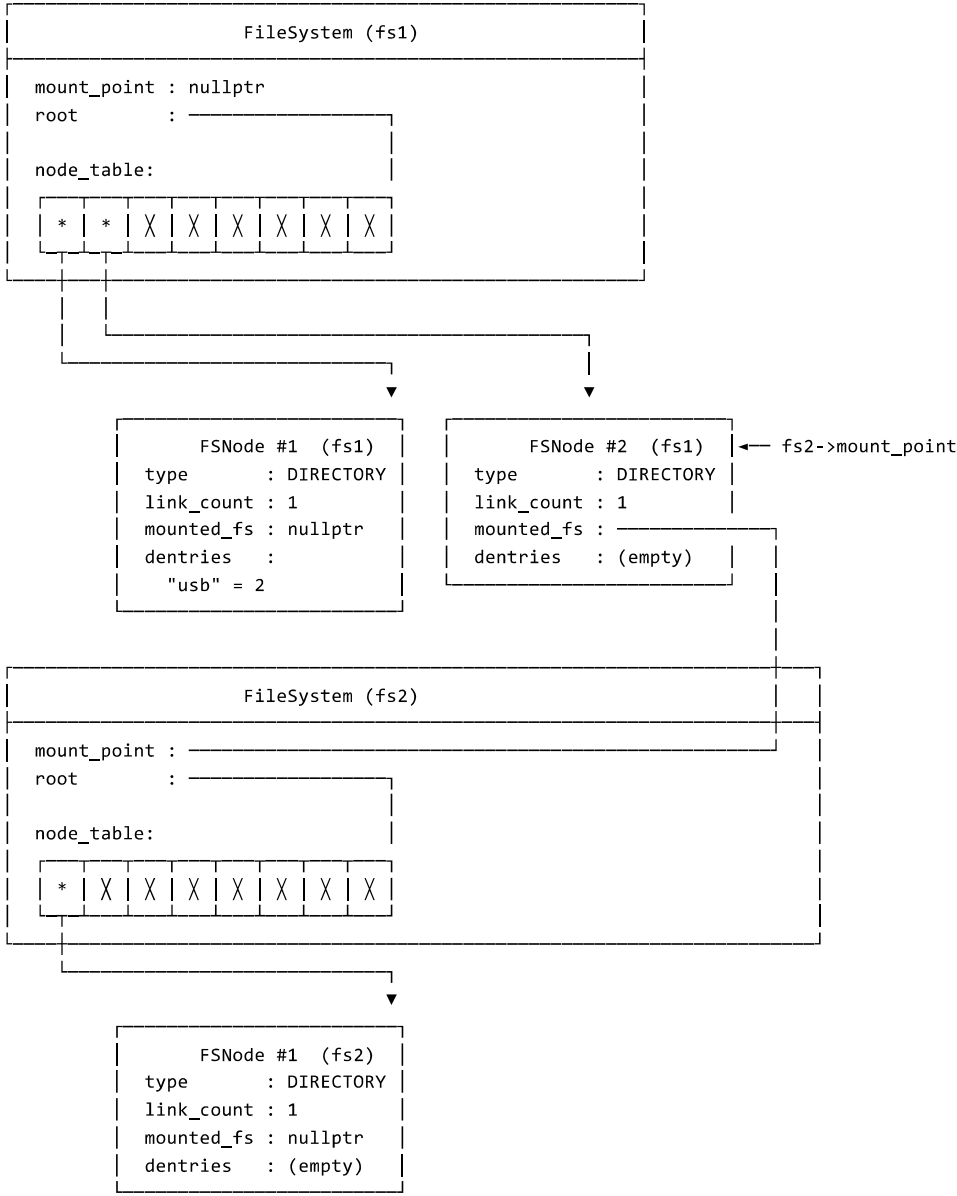
想像插入一支 `USB`，它有自己的檔案系統，插上之後你可以在指定目錄底下看到它的內容。



在 SimpleFS 中，掛載就是把一個 FileSystem (fs2) 的根目錄「接」到另一個 FileSystem 的某個目錄上 (fs1->usb)，形成連結：

- fs2->mount_point → 指向 fs1 的某個目錄 FSNode
- 該目錄 FSNode 的 mounted_fs → 指回 fs2

值得注意的是真實系統並不這麼實現，這只是一個簡化過後的結果。



系統規則

在開始實作之前，先了解這個系統的設計原則。整個作業的核心是理解 **Pointer** 的生命週期。



多個物件之間的關係

關係	說明	例子
管理	A 建立 B，A 消失時 B 跟著消失	FileSystem 管理所有 FSNode，因此 FileSystem 消失時，FSNode 也需要跟著消失。
引用	A 使用 B，但不負責 B 的生死	FileHandle 引用 FSNode
連結	A 與 B 互相持有對方的 pointer，但各自獨立存活	FSNode::mounted_fs ↔ FileSystem::mount_point

FSNode 的生命週期

FSNode 由 FileSystem 統一建立與釋放，從 allocNode 開始，但釋放分成**安全釋放**以及**強制釋放**：

- 安全釋放：
 - 當該 FSNode 沒有人在使用時才能使用 freeNode 安全釋放，否則會變成 dangling pointer。
 - freeNode 需檢查安全釋放的條件，不滿足則回傳 false，條件如下：
 - 當沒有 dentry 指著這個 FSNode (用 link_count == 0 判斷)
 - 當沒有 FileHandle 正在開著這個 FSNode (用 open_count == 0 判斷)
- 強制釋放：
 - FileSystem destructor 時，因為管理者消失，因此應該強制釋放所有 FSNode。

FileHandle 的生命週期

FileHandle 由使用者建立，用來開啟並讀寫一個 FSNode。建構時 open_count++，呼叫 close() 時 open_count--。若 delete 前忘記呼叫 close()，destructor 應自動補上。

FileHandle 不擁有它指向的 FSNode，不能 delete 它。

FileSystem 的生命週期

FileSystem 建構時會自動建立根目錄 (root)，並初始化 node_table。從這個時間點起，所有透過 allocNode 建立的 FSNode 都由它管理。

delete FileSystem 時，它必須釋放所有自己建立的 FSNode，包含根目錄。唯一的例外是 mount_point——那是指向另一個 FileSystem 的 FSNode，不屬於它，不能 delete。

計數的變化時機

事件	影響
一個 dentry 指向某個 FSNode	link_count 加一
一個 dentry 被移除	link_count 減一
FileHandle 開啟某個 FSNode	open_count 加一
FileHandle 關閉	open_count 減一

題目需求

整個題目有五個需求，配分如下。

- 需求 1：我想要一個檔案系統的根目錄
 - 重點函式：FileSystem constructor、destructor、allocNode



- 配分：20 分
- 需求 2：我想建立資料夾和檔案。
 - 重點函式：addDentry、findDentry、getNode
 - 配分：20 分
- 需求 3：我想開啟一個檔案來讀寫，讀完要能關掉。
 - 重點函式：FileHandle constructor、destructor、open、close
 - 配分：20 分
- 需求 4：我想刪掉不再需要的檔案，但我怕刪錯。
 - 重點函式：removeDentry、freeNode
 - 配分：20 分
- 需求 5：我想把另一個檔案系統掛進來，像插 USB 一樣，不用了再拔掉。
 - 重點函式：mount、unmount，更新
addDentry、removeDentry、findDentry、listDentries
 - 配分：20 分

需求一：建立檔案系統

提示：完成所有 [TODO 1] 標籤

檔案系統的起始點稱為根目錄，就像 Linux 的 / 一樣，呼叫 FileSystem 建構時即應該存在。根目錄同樣是一個 FSNode，因此需要在建構時新增一個 FSNode 設定為根目錄。

重點函式：

- FileSystem::FileSystem：配置 node_table，呼叫 allocNode 建立根目錄，並將 root 的 link_count 設為 1
- FileSystem::allocNode：在 node_table 掃描空槽，配置並回傳新的 FSNode；已滿時回傳 nullptr
- FileSystem::~FileSystem：強制釋放所有 FSNode 及 node_table，不論 link_count 或 open_count

測試範例：

```
#include <iostream>
#include "FileSystem.h"

int main () {
    FileSystem* fs1 = new FileSystem(8);
    fs1->getRoot()->printInfo();
    delete fs1;
}
```

通過標準：

HW3 FSNode.cpp FileHandle.cpp FileSystem.cpp 1

成功範例：



```
=====
Homework 3: SimpleFS
=====
Starting Evaluation... (Target: Subtask 1)

Subtask 1 (FileSystem & allocFSNode): PASSED

=====
```

失敗範例：

```
=====
Homework 3: SimpleFS
=====
Starting Evaluation... (Target: Subtask 1)

Subtask 1 (FileSystem & allocFSNode): FAILED
failed: root link_count must be 1

=====
```

需求二：新增資料夾和檔案在根目錄中

提示：完成所有 [TODO 2] 標籤

有了根目錄之後，我們要能在裡面建立資料夾和檔案，並且能用名字找到它們。

1. 加入目錄：呼叫 `addDentry` 把一個 `FSNode` 加入某個目錄，建立「名稱 → `FSNode`」的對應。
同一個目錄下**不能有重複的名稱**，若已存在同名 `dentry` 則新增失敗。
2. 查詢目錄中特定名稱：呼叫 `findDentry` 用名稱找到對應的 `node id`，再到 `fs` 找到實際的 `FSNode pointer`。

重點函式：

- `FSNode::addDentry`：在 `dentry table` 找空槽填入，並更新 `target` 的 `link_count`；名稱已存在或 `table` 已滿時回傳 `false`
- `FSNode::findDentry`：找到名稱對應的 `FSNode*`；找不到回傳 `nullptr`
- `FileSystem::getNode`：依 `node_id` 在 `node_table` 中找到對應的 `FSNode*`；找不到回傳 `nullptr`

測試範例：

```
#include <iostream>
#include "FileSystem.h"

int main () {
    FileSystem* fs1 = new FileSystem(8);
    FSNode* home   = fs1->allocNode(FSNodeType::DIRECTORY);
    FSNode* readme = fs1->allocNode(FSNodeType::FILE, "Hello NCKU");

    fs1->getRoot()->addDentry("home", home);
    home->addDentry("readme.txt", readme);
    home->listDentries(fs1);

    FSNode* found = home->findDentry("readme.txt", fs1);

    delete fs1;
}
```

通過標準：

HW3 FSNode.cpp FileHandle.cpp FileSystem.cpp 2

需求三：檔案讀寫

提示：完成所有 [TODO 3] 標籤

我們成功新增檔案，但要怎麼讀寫它？定義一個 `FileHandle` 來開啟檔案、編輯檔案。

想像一個情境，如果我正在編輯一個檔案，另一個人把它從目錄裡移除。如果系統立刻釋放這個檔案的記憶體，這個開啟的檔案就會指向已經被釋放的記憶體，讀寫會出問題。這是一種 `dangling pointer`。

因此當 `FileHandle` 開啟一個檔案時，該檔案代表的 `FSNode` 需要維護一個 `open_count` 變數，紀錄該檔案有人開啟。

重點函式：

- `FileHandle::FileHandle`：標記為開啟，更新 `open_count`
- `FileHandle::~~FileHandle`：若尚未關閉，自動呼叫 `close()`；不能 `delete node`
- `FileHandle::open`：若已開啟某個檔案，先關閉再開啟新的
- `FileHandle::close`：標記為關閉，更新 `open_count`

測試範例：

```
#include <iostream>
#include "FileSystem.h"
#include "FileHandle.h"

int main () {
    FileSystem* fs1 = new FileSystem(8);
    FSNode* home = fs1->allocNode(FSNodeType::DIRECTORY);
    FSNode* doc = fs1->allocNode(FSNodeType::FILE, "initial content");
    home->addDentry("doc.txt", doc);

    FileHandle* fh = new FileHandle(doc);

    std::cout << fh->read() << "\n";
    fh->write("updated content");
    std::cout << fh->read() << "\n";

    fh->close();
    delete fh;

    doc->printInfo();
    delete fs1;
}
```

通過標準：

HW3 FSNode.cpp FileHandle.cpp FileSystem.cpp 3

需求四：刪除檔案

提示：完成所有 [TODO 4] 標籤

接下來到最危險的一步了，我們可不能隨意刪除檔案，一不小心刪錯就糟糕了，切記要小心！請謹慎的思考何時才能夠真正地將 `FSNode` 釋放掉，應該由誰釋放 `FSNode`。

重點函式：

- `FSNode::removeDentry`：在 `dentry table` 找到對應名稱的 `slot`，清空它，並更新 `link_count`；找不到回傳 `false`。只移除對應關係，不釋放 `FSNode` 本身。



- `FileSystem::freeNode` : `link_count == 0` 且 `open_count == 0` 時才能釋放，並清空 `node_table` 中的槽位；否則回傳 `false`。

測試範例：

```
#include <iostream>
#include "FileSystem.h"
#include "FileHandle.h"

int main () {
    FileSystem* fs1 = new FileSystem(8);
    FSNode* home   = fs1->allocNode(FSNodeType::DIRECTORY);
    FSNode* readme = fs1->allocNode(FSNodeType::FILE, "Hello");

    home->addDentry("readme.txt", readme);
    home->removeDentry("readme.txt", fs1);
    fs1->freeNode(readme->getNodeId());
    delete fs1;
}
```

通過標準：

```
HW3 FSNode.cpp FileHandle.cpp FileSystem.cpp 4
```

需求五：掛載另一個檔案系統

提示：完成所有 [TODO 5] 標籤

看起來基本功能都實現的差不多了，但是我還想要可以像插其他 `usb` 這樣一個很酷的功能。

掛載後，對掛載點目錄的操作行為如下：

操作	行為
<code>listDentries</code>	轉向顯示 <code>mounted fs</code> 根目錄的內容
<code>findDentry</code>	轉向到 <code>mounted fs</code> 根目錄查詢
<code>addDentry</code>	回傳 <code>false</code> （禁止， <code>fs</code> 歸屬不明）
<code>removeDentry</code>	回傳 <code>false</code> （禁止， <code>fs</code> 歸屬不明）

若要在 `mounted fs` 新增或刪除，應直接操作 `fs2->getRoot()`。

重點函式：

- `FileSystem::mount`：建立 `mount_point ↔ mounted_fs` 雙向連結；此 `FileSystem` 已掛載、`mountpoint` 為 `nullptr` 或非 `DIRECTORY` 時回傳 `false`
- `FileSystem::unmount`：清除雙向連結；未掛載時回傳 `false`
- 更新 `addDentry`、`removeDentry`、`findDentry`、`listDentries`：加入掛載狀態的判斷

測試範例：



```
#include <iostream>
#include "FileSystem.h"
#include "FileHandle.h"

int main () {
    FileSystem* fs1 = new FileSystem(8);
    FSNode* usb    = fs1->allocNode(FSNodeType::DIRECTORY);
    fs1->getRoot()->addDentry("usb", usb);

    FileSystem* fs2 = new FileSystem(8);
    fs2->mount(usb);
    usb->listDentries(fs1);
    fs2->unmount();

    delete fs2;
    delete fs1;
}
```

通過標準：

HW3 FSNode.cpp FileHandle.cpp FileSystem.cpp 5

注意事項！

1. 提供的 FSNode.h, FileSystem.h, FileHandle.h, FSNodeType.h 不應修改。
2. 自行建立 FSNode.cpp, FileHandle.cpp, FileSystem.cpp，實作 .h 中標有 [TODO x] 的所有函式。
3. printInfo、listDentries、listAll 不計入評分，可自由實作。
4. 呼叫 HW3 FSNode.cpp FileHandle.cpp FileSystem.cpp [subtask_number] 確認需求是否完成。
5. 完成所有需求後，執行 HW3 FSNode.cpp FileHandle.cpp FileSystem.cpp 出現的總分，即為該作業分數。
6. 若以上題目有任何問題，歡迎在 DC 提出！

全部通過：

\$ HW3 FSNode.cpp FileHandle.cpp FileSystem.cpp

```
=====
Homework 3: SimpleFS
=====
Starting Evaluation...

Subtask 1 (FileSystem & allocFSNode): PASSED [20/20 pts]
Subtask 2 (dentry operations): PASSED [20/20 pts]
Subtask 3 (FileHandle): PASSED [20/20 pts]
Subtask 4 (freeFSNode): PASSED [20/20 pts]
Subtask 5 (mount & unmount): PASSED [20/20 pts]

=====
Final Score: 100 / 100
=====
```



程式模板

FSNodeType.h

```
#pragma once

enum class FSNodeType {
    FILE,
    DIRECTORY
};
```



FSNode.h

```
#pragma once
#include "FSNodeType.h"

class FileSystem;

#define MAX_DENTRIES 16
#define MAX_NAME_LEN 14

class FSNode {
private:
    int node_id;
    FSNodeType type;
    int link_count;
    int open_count;
    char content[256];

    int dentry_inums[MAX_DENTRIES];
    char dentry_names[MAX_DENTRIES][MAX_NAME_LEN + 1];

    FileSystem* mounted_fs;

public:
    // [TODO 1]
    FSNode(int node_id, FSNodeType type, const char* content = "");
    // [TODO 1]
    ~FSNode();

    // [TODO 1]
    int getId() const;
    // [TODO 1]
    FSNodeType getType() const;
    // [TODO 1]
    int getLinkCount() const;
    // [TODO 1]
    int getOpenCount() const;
    // [TODO 1]
    const char* getContent() const;
    // [TODO 1]
    FileSystem* getMountedFs() const;

    // [TODO 1]
    void setContent(const char* new_content);
    // [TODO 1]
    void setMountedFs(FileSystem* fs);

    // [TODO 1]
    void incrementLinkCount();
    // [TODO 1]
    void decrementLinkCount();
    // [TODO 1]
    void incrementOpenCount();
    // [TODO 1]
    void decrementOpenCount();

    // [TODO 2]
    // [TODO 5]
    bool addDentry(const char* name, FSNode* target);

    // [TODO 4]
    // [TODO 5]
    bool removeDentry(const char* name, FileSystem* fs);

    // [TODO 2]
    // [TODO 5]
    FSNode* findDentry(const char* name, FileSystem* fs) const;

    // [TODO 5]
    void listDentries(FileSystem* fs) const;

    void printInfo() const;
};
```



FileSystem.h

```
#pragma once
#include "FSNode.h"

class FileSystem {
private:
    FSNode** node_table;
    int capacity;
    int node_count;
    int next_number;
    FSNode* root;
    FSNode* mount_point;

public:
    // [TODO 1]
    //
    // Steps:
    // 1. Allocate an array of FSNode* of size capacity, store in node_table
    //    Initialize every slot to nullptr
    // 2. Call allocFSNode() to create root
    //    - update root pointer
    //    - update link_count
    FileSystem(int capacity = 16);

    // [TODO 1]
    ~FileSystem();

    // [TODO 1]
    //
    // Steps:
    // 1. Find the first nullptr slot in node_table
    //    If table full -> Return nullptr
    // 2. new FSNode()
    //    Store in the found slot
    //    next_number++, node_count++
    // 3. Return the node
    FSNode* allocNode(FSNodeType type, const char* content = "");
    // [TODO 4]
    bool freeNode(int node_id);

    // [TODO 5]
    bool mount(FSNode* mountpoint);
    // [TODO 5]
    bool unmount();
    // [TODO 5]
    FSNode* getMountPoint() const;

    // [TODO 2]
    FSNode* getNode(int node_id) const;
    // [TODO 1]
    FSNode* getRoot() const;

    void listAll() const;
};
```



FileHandle.h

```
#pragma once
#include "FSNode.h"

class FileHandle {
private:
    FSNode* node;
    bool is_open;
    int offset;

public:
    // [TODO 3]
    FileHandle(FSNode* node);
    // [TODO 3]
    ~FileHandle();

    // [TODO 3]
    void open(FSNode* node);
    // [TODO 3]
    void close();

    void seek(int pos);
    int tell() const;
    bool isOpen() const;
    const char* read() const;
    void write(const char* data);
    void printInfo() const;
};
```

FSNode.cpp (部分提供)

```
#include "FSNode.h"
#include "FileSystem.h"
#include <cstring>
#include <iostream>

void FSNode::listDentries(FileSystem* fs) const {
    if (type != FSNodeType::DIRECTORY)
        return;

    // [TODO 5]
    std::cout << "List FSNode #" << node_id << " (DIRECTORY):" << "\n";
    for (int i = 0; i < MAX_DENTRIES; i++) {
        if (dentry_inums[i] != 0) {
            FSNode* target = fs->getNode(dentry_inums[i]);
            std::cout << "  \"" << dentry_names[i] << "\" -> FSNode #"
                << dentry_inums[i] << " ("
                << (target && target->getType() == FSNodeType::DIRECTORY
                    ? "DIRECTORY"
                    : "FILE")
                << ")\n";
        }
    }
}
```